SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks:50**

Mode: In Class **Hours : 60 mts**

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I E.Ragul / 192571032 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Ragul

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results(15)	Total Score (out of 100)
1	/4	/4	/4	/4	/4	
2	/4	/4	/4	/4	/4	
3	/4	/4	/4	/4	/4	
4	/4	/4	/4	/4	/4	
5	/4	/4	/4	/4	/4	
OVERALL RUBRICS VALUE						
	/4	/4	/4	/4	/4	
	/20	/20	/25	/20	/15	/100

Annexure A

ANALYTICAL PROBLEM SOLVING

1	Trace the processing of the input expression $P = (a * b) + (b * c) * 2$ through all phases of a compiler. a) Identify and list all tokens generated during lexical analysis. b) Construct the syntax tree during the parsing phase. c) Generate the intermediate code for the given expression. d) Apply suitable optimizations to the intermediate code. e) Explain the significance of the final target code prod
2	For each of the following Regular Expressions, construct the language. Write any FIVE valid strings for each Regular Expression. a)$(\varepsilon+aa^*)^*$ b)$(a^*+b^*)^* abb$ c)$(a^*b^*)^* aba\,(a^* + b^*)^*$ d)$(a+ab)^*$ e) $(aba)^* + (a^*b^*)^*$

3	i)Find the number of tokens in the following C statement is
	main( )
	{
	int a=10;
	char b ="abc";
	in tc = 30;
	chard = "xyz";
	in /* comment */ t m = 40.5;
	}
	ii)Identify the lexical errors in the following C statement is
	voidmain()
	{
	int x=10, y=20;
	char * a;
	a= &x;
	x= 1xab;
	}

4	**Find a regular expression corresponding to each of the following subsets of $\{0, 1\}^*$:** i) The language of all strings that have an even number of 0's. ii) The language of all strings that contain at least one 1. iii) The language of all strings in which both the number of 0's and the number of 1's are odd. iv) The language of all strings that start with 1 and end with 0. v) The language of all strings that do not contain consecutive 1's.

Consider a lexical analyzer for a simplified programming language. The language consists of the following tokens:

1. Keywords: if, else, while, return, int, float
2. Operators: +, -, *, /, =, ==, <, >
3. Separators: (,), {, }, ;, ,
4. Identifiers: A sequence of letters followed by a sequence of digits, e.g., var123
5. Integer Literals: A sequence of digits, e.g., 12345
6. Floating-point Literals: A sequence of digits followed by a dot and another sequence of digits, e.g., 123.45

The lexical analyzer needs to tokenize the following input string:
int var123 = 12345; float var456 = 123.45; if (var123 == var456) { return; }

Questions:

a) How many tokens are there in the given input string?

b) Provide a list of tokens categorized by their types (keywords, operators, separators, identifiers, integer literals, floating-point literals).

c) Assuming a finite state machine (FSM) is used for lexical analysis, estimate the number of state transitions required to tokenize the given input string. Provide your reasoning based on the transitions needed for each token type.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method

ıtion Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1. Trace The Processing of the Input

Given:

$$P = (a * b) + (b * c) * 2$$

9/2/00

A) Lexical Analysis

The expression produces 15 tokens

P | = | (| a | * | b |) | + | (| b | * | c |) | * | 2

→ P, a, b, c → Identifiers
→ =, +, * → Operators
→ (,) → seperators
→ 2 → Integer literal

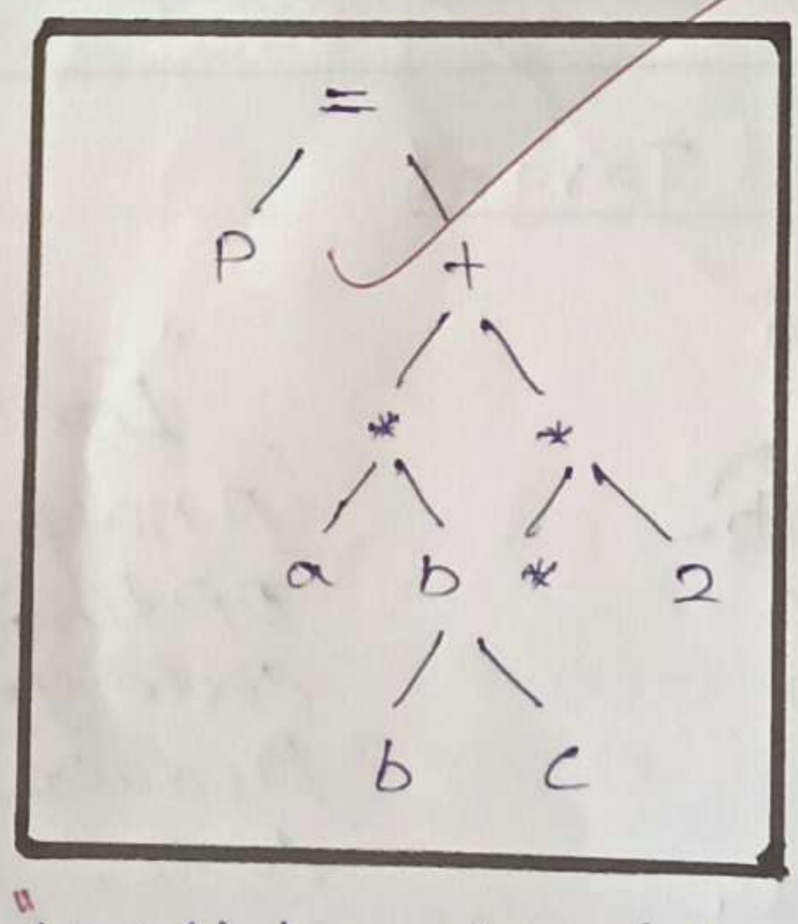
TOTAL = 15 Tokens

B) Syntax Analysis

Using Operator precedence

$$P = (a * b) + ((b * c) * 2)$$

Syntax Tree:



"MULtiplication is Performed before Addition"

c) Intermediate Code

Using three Address Code

$t_1 = a * b$
$t_2 = b * c$
$t_3 = t_2 * 2$
$t_4 = t_1 + t_3$
$P = t_4$

d) Code optimization

The unnecessary temporary t_4 is eliminated

$t_1 = a * b$
$t_2 = b * c$
$t_3 = t_2 * 2$
$P = t_1 + t_3$

e) Significance of Final Target Code

Simplified Target

LOAD R1, a
MUL R1, b
LOAD R2, b
MUL R2, c
MUL R2, 2
ADD R1, R2
STORE P, R1

"Lexical analysis, Syntax Analysis, Intermediate, Optimization, target code generation to Produce efficient executable code!"

2. Construct the Language and write Five valid Strings

a) $(\varepsilon + aa^*)^*$

Since:

$$aa^* = \{a, aa, aaa, \ldots\}$$

and including ε, the expression generates all strings containing only a.

Language: $L = \{a\}^*$

Five valid strings:

1) ε	2) a	3) aa	4) aaa	5) aaaa

b) $(a^* + b^*)^* abb$

Here, $(a^)^*$ can generate any combination of a and b

Therefore: The language consists of ALL strings that end with abb

1) abb	2) aabb	3) babb
	4) ababb	5) aaabb

c) $(a^)^ + b^*)^*$

$$(a^* b^*)^* = (a + b)^*$$

Therefore:

The expression represents all strings containing aba a substring

Five valid strings:

1) aba	2) aaba	3) abaa	4) baba	5) ababa

D) (a+ab)*

Each block is either a or ab

So the Language contains strings formed by repeatedly choosing a or ab.

Five valid strings:

1> ε	2> a	3> ab	4> aa	5> aab

e) (aba)^* + (a^)^*

Since: (a*b*)* = (a+b)*

It can generate all strings over {a, b}

Therefore:

L = (a+b)*

Five valid strings:

1> ε	2> a	3> b	4> ab	5> ba

3. Lexical Analysis

1) Find the number of tokens in the c statement

Given:

```
main()
{
int a=10;
Char b ="abc"
int c = 30;
Ch ar d = "XYZ";
in/*comment*/t m=40.5;
}
```

Step 1: Remove spaces and comments

- in t → inand t
- ch ar → ch and ar
- in /* Comment */ t → in& and t
- /* Comment */ → Ignored

Step 2: Token Count

NO	Token	TyPe
1	main	Identifier
2	(	Separator
3	)	Separator
4	{	Separator
5	int	keyword
6	a	Identifier
7	=	OPerator
8	10	Integer
9	;	Seperator
10	char	keyword
11	b	Identifier
12	=	Operator
13	"abc"	Stringliteral
14	;	Seperator
15	in	Identifier
16	t	Identifier
17	c	Identifier
18	=	OPerator
19	30	Integer
20	;	Seperator
21	ch	Identifier
22	ar	Identifier
23	d	Identifier
24	=	operator
25	xyz	Stringliteral
26	;	Separator
27	in	Identifier
28	t	Identifior
29	m	Identifier
30	=	operator
31	40.5	Floating literal
32	;	Seperator
33	}	Seperator

Total number of Tokens = 33

ii) Identify the Lexical Errors

Given:

```
Void main()
{
 int x = 10, y = 20
 Char *a
 a = &x;
 X = 1xab;
}
```

The error is:

X = 1xab;

Because, 1xab is an invalid numeric taken because Letters occurs in an invalid position within the number

---

4. Regular Expressions for {0,1}*

i) Even number of 0s

1* (01*01*)*

Ex: ε, 1, 11, 00, 1001

ii) Atleast one 1

(0+1)* 1 (0+1)*

This guarantees that at Least one 1 occurs

Ex: 1, 01, 10, 101, 00100

a) Number of tokens

Tokens are:

int
Var123
=
12345
;
Float
Var456
=
123.45
;
if
(
Var123
==
var456
)
{
return
;
}

Total = 20 tokens

b) Categorization

a) keywords - Total = 4

int, Float, if, return

b) operator - Total = 3

=, =, ==

iii) Both number of 0s and 1s are odd

Let: E = [00+11]

M = [01+10]

A suitable regular expression is:

(00+11)*(01+10)

((00+11)*(01+10)(00+11)*(01+10))*(00+11)*

Ex: 01, 10, 0011, 0110, 1001

iv) Starts with 1 and ends with 0

1(0+1)*0

Ex: 10, 110, 100, 1010, 1110

v) Does not contain consuctive 1s

(0+10)*(ε+1)

Ex: ε, 0, 1, 10, 101, 1010

The expression never permits

5. Lexical Analyzer

Given Input:

int var123 = 12345; float var456 = 123.45;

if(var123==var456){return;}

d) Identifiers - Total = 4

Var 123, Var 456, Var 123, Var 456

e) Integer literal - Total - 1

12345

F) floating-point - Total - 1

123.45

∴ Total tokens = 20

c) FSM state Transitions

q0 → Start

q1 → Identifier

qN → Integer

qF → Fraction

Total Transitions = 61

Final result:

The Lexical analyzer recoghizes 20 tokens classifier then correctly and user FSM transitions to Identify keywords, Identifers, numbers, operators and seperators